Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference

Deep Learning (CMP030-L043-0) Part 1 Assessment Report

Collins Chikaodi Collins A00070784

Word Count: 2000

Table of Contents

1 .Summary

The paper **“Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference”** investigates an often-overlooked challenge in deploying Large Language Models (LLMs): deterministic decoding does not always guarantee deterministic outputs. Although decoding strategies such as greedy decoding are designed to produce identical responses for identical prompts, the authors demonstrate that numerical variations introduced by floating-point arithmetic and parallel GPU execution can still alter the generated text. These variations arise because floating-point operations are non-associative, meaning that different execution orders during matrix computations may produce slightly different numerical values. As these differences propagate through multiple Transformer layers, they can modify token probability distributions and ultimately influence token selection during inference [1], [2].

To address this problem, the paper introduces **LayerCast**, a numerical stabilisation technique that applies consistent precision casting throughout the inference pipeline to minimise hardware-induced numerical variation. The proposed approach is evaluated across multiple inference configurations, including different batch sizes and GPU counts, demonstrating significant improvements in output reproducibility while maintaining comparable model performance. The paper makes an important contribution by highlighting that reproducibility in modern LLMs depends not only on deterministic decoding algorithms but also on the underlying numerical behaviour of deep learning hardware and software. Its findings have practical implications for research reproducibility, benchmarking, and the reliable deployment of AI systems in domains where consistent and repeatable model outputs are essential [1], [3].

2. Critical Appraisal

2.1 Theoretical Foundations

The paper is founded on the interaction between Transformer architectures, floating-point arithmetic, and parallel GPU computation. Transformer models perform billions of numerical operations during inference, with self-attention, matrix multiplication, and softmax normalisation forming the core computational components [4], [12]. Although deterministic decoding strategies, such as greedy decoding, eliminate randomness introduced through sampling, they do not eliminate numerical variation arising from floating-point computation. Consequently, reproducibility depends not only on the decoding algorithm but also on the numerical behaviour of the underlying hardware and software stack.

A major strength of the paper is its identification of floating-point non-associativity as the principal source of inference nondeterminism. According to the IEEE 754 floating-point standard, arithmetic operations executed in different orders may produce slightly different results because of rounding behaviour [16]. In large-scale GPU environments, reductions and matrix operations are distributed across thousands of processing cores, meaning execution order can change depending on hardware configuration or workload scheduling. Although the resulting numerical differences are extremely small, repeated propagation through multiple Transformer layers may alter token probability distributions sufficiently to change the final generated output [1].

The theoretical discussion is technically accurate and demonstrates a strong understanding of numerical stability in deep learning systems. However, the analysis remains largely descriptive. The argument would be strengthened by incorporating a formal sensitivity analysis showing how small perturbations introduced during floating-point computation propagate through attention layers and influence token-probability rankings. Such an analysis would provide stronger theoretical justification for the observed behaviour while establishing a clearer relationship between numerical precision and inference reproducibility.

2.2 Methodology Critique

The experimental methodology represents one of the strongest aspects of the study because it successfully isolates numerical nondeterminism from stochastic variation. By employing deterministic decoding with fixed prompts and model parameters, the experimental design ensures that any observed differences originate from numerical computation rather than randomness associated with sampling strategies. This provides a reliable framework for investigating inference reproducibility and enables the numerical sources of variation to be examined independently.

Another significant strength is the evaluation across different GPU counts and batch sizes. Since modern LLMs are commonly deployed in distributed environments, demonstrating that hardware configuration alone can influence generated outputs highlights an important practical limitation of current inference pipelines. The inclusion of LayerCast as a mitigation strategy further strengthens the study by showing that improvements in numerical consistency can be achieved without substantially affecting model performance.

Despite these strengths, the methodology exhibits several limitations. The evaluation focuses primarily on comparisons with conventional inference pipelines while providing limited comparison against alternative numerical stabilisation techniques. Existing approaches, including deterministic CUDA execution, reproducible reduction algorithms, and framework-level deterministic controls, receive little consideration. Without broader comparative evaluation, it remains difficult to determine whether LayerCast represents the most effective solution or simply one possible implementation.

The experimental evaluation could also be strengthened through greater diversity of benchmark tasks. Although the reported experiments demonstrate the existence of numerical nondeterminism, they provide limited evidence that the findings generalise across reasoning, summarisation, code generation, and conversational tasks. Furthermore, the ablation studies remain relatively limited. Identifying the specific Transformer components most responsible for numerical instability would have provided greater insight into the origin of nondeterministic behaviour and supported the development of more targeted mitigation strategies.

2.3 Comparison with State of Art

While the paper makes a valuable contribution to understanding inference reproducibility, its engagement with the broader state-of-the-art literature is relatively limited. Previous research has investigated numerical stability through mixed-precision optimisation, deterministic execution modes, and framework-level reproducibility controls implemented within deep learning libraries such as PyTorch [5], [10]. These studies

aim to reduce numerical variation by improving precision management or enforcing deterministic execution across hardware platforms.[7][4]

LayerCast differs from these approaches by targeting consistent numerical casting throughout the inference pipeline rather than modifying model architecture or training procedures. This represents a practical contribution because it addresses inference reliability without requiring retraining of the model. However, the absence of direct comparison with existing numerical stabilisation strategies makes it difficult to assess the relative effectiveness of the proposed approach. Incorporating comparative experiments against established reproducibility techniques would provide stronger evidence that LayerCast offers measurable advantages over existing methods while positioning the work more clearly within the wider deep learning literature.[3][6]

2.4 Limitation and Risk

Although LayerCast improves numerical consistency during inference, several practical challenges remain. Increasing numerical precision inevitably introduces additional computational cost, creating a trade-off between reproducibility and inference efficiency. This consideration becomes increasingly important for production-scale LLM deployments where latency, throughput, and memory utilisation directly influence operational performance.

Complete determinism also remains difficult to achieve because numerical behaviour is affected by multiple factors beyond precision casting, including GPU architecture, compiler optimisations, CUDA implementations, and runtime scheduling. Consequently, the proposed approach should be viewed as reducing nondeterminism rather than eliminating it entirely.

From an application perspective, inconsistent outputs present important reliability concerns. Scientific benchmarking, model auditing, and safety-critical applications all depend upon reproducible model behaviour. If identical prompts produce different responses under equivalent inference settings, confidence in AI-assisted decision-making may be reduced. Consequently, improving numerical determinism represents not only a technical optimisation problem but also an important requirement for trustworthy and responsible deployment of large language models.[3][9]

3.Proposal for improvement

3.1 Proposal for improvement

The critical appraisal identified that, although the selected paper successfully demonstrates the existence of numerical nondeterminism during Large Language Model (LLM) inference, its evaluation remains limited in scope.[4] The experiments primarily demonstrate that output variation exists under different hardware configurations, but provide comparatively little quantitative analysis of inference performance, numerical reproducibility, or the relationship between numerical precision and computational efficiency. Consequently, the proposed improvement focuses on extending the evaluation methodology rather than introducing another numerical mitigation algorithm.

This study proposes the development of a **Deterministic LLM Inference Evaluation Framework**, designed to systematically investigate how numerical precision influences inference reproducibility and performance under controlled experimental conditions.[6] Rather than modifying the internal architecture of a language model, the framework provides a repeatable experimental environment capable of evaluating inference consistency across multiple execution settings while maintaining identical prompts and deterministic decoding parameters.

The framework employs an open-source Transformer-based language model and executes repeated inference experiments using identical inputs while controlling generation parameters, precision settings, and execution environment. For each experiment, inference outputs, execution time, memory utilisation, and numerical consistency are recorded and analysed using quantitative performance metrics. The collected results provide a structured basis for assessing how numerical precision influences both computational efficiency and output reproducibility.

By shifting the emphasis from proposing an additional mitigation technique to constructing a comprehensive evaluation framework, the proposed improvement directly addresses the limitations identified in the critical appraisal. The framework enables reproducible experimentation, facilitates comparison across different inference configurations, and provides empirical evidence that can guide future research into numerical stability within modern LLM deployment environments.

3.2 Technical Justification

The proposed evaluation framework is motivated by the observation that reliable AI systems require both reproducible outputs and transparent experimental evidence.[6] Although the selected paper demonstrates that floating-point arithmetic can introduce numerical variation during inference, reproducibility cannot be fully assessed without systematic experimentation across different computational settings. Consequently, an experimental framework provides greater practical value than evaluating only a limited number of inference scenarios.[1]

A framework-based approach also aligns with established software engineering principles by separating experimentation from model development. Instead of modifying the architecture of an existing language model, the framework provides an independent environment for evaluating inference behaviour under controlled conditions. This improves repeatability because identical prompts, decoding parameters, and execution settings can be reused across multiple experiments while individual variables, such as numerical precision, are altered independently.[4]

The proposed approach also supports quantitative evaluation, an aspect identified as a limitation within the selected paper. In addition to measuring output consistency, the framework records inference time and memory utilisation, allowing computational cost to be analysed alongside reproducibility. This enables trade-offs between numerical stability and inference efficiency to be investigated more comprehensively than qualitative observations alone.[9]

From an engineering perspective, the framework is intentionally lightweight and compatible with widely adopted deep learning libraries. Modern frameworks such as PyTorch and Hugging Face Transformers provide deterministic decoding, precision management, and hardware acceleration, allowing reproducible experiments to be implemented without modifying the underlying model architecture. This improves portability while enabling the evaluation methodology to be applied to other open-source LLMs beyond the model selected for this study.

3.3 Technical Design and Implementation Plan

The proposed framework consists of four interconnected components that together provide a structured pipeline for evaluating numerical nondeterminism during LLM inference.

The first component is the **model execution module**, responsible for loading the selected Transformer model and tokenizer while establishing a deterministic execution environment. Deterministic decoding is

enforced by disabling stochastic sampling techniques and maintaining identical generation parameters throughout every experiment. This ensures that any observed differences originate from numerical computation rather than probabilistic decoding strategies.

The second component is the **experimental evaluation module**, which performs repeated inference using identical prompts under controlled execution conditions. Separate experiments evaluate different numerical precision configurations while maintaining identical inputs and decoding parameters. This enables the influence of floating-point precision on inference behaviour to be examined systematically without introducing additional experimental variables.

The third component is the **performance monitoring module**, which records computational metrics generated during each experiment. These include inference execution time, GPU memory utilisation, generated outputs, and precision configuration. Collecting these measurements allows reproducibility to be evaluated alongside computational efficiency, providing a more comprehensive assessment of deployment performance than output comparison alone.

The final component is the **analysis and reporting module**, responsible for aggregating experimental results and generating visualisations that summarise inference behaviour. Statistical summaries, performance graphs, and exported datasets provide a reproducible record of every experiment while enabling direct comparison between different execution settings. The modular architecture also supports future extension, allowing additional language models, evaluation metrics, or hardware platforms to be incorporated with minimal modification.

Overall, the proposed framework provides a structured engineering solution that translates theoretical discussions of numerical nondeterminism into measurable experimental evidence, thereby supporting more rigorous evaluation of reproducibility in modern language model inference.

3.4 Expected Outcomes

The proposed evaluation framework is expected to provide a more comprehensive understanding of numerical nondeterminism than qualitative observation alone. By systematically measuring inference reproducibility, computational performance, and resource utilisation, the framework enables the relationship between numerical precision and inference behaviour to be examined using objective experimental evidence.

A further expected outcome is improved research reproducibility. Because every experiment is performed under controlled deterministic conditions, future researchers can repeat the evaluation using identical prompts, generation parameters, and experimental procedures.[4][8]This contributes to greater transparency and supports fair comparison between alternative numerical stabilisation techniques.

The framework is also expected to demonstrate that numerical precision influences not only output consistency but also computational performance.[5][2[Measuring inference time and memory utilisation alongside reproducibility provides a balanced assessment of the trade-offs associated with deterministic inference, allowing researchers to evaluate whether improvements in numerical stability justify any additional computational cost.[10]

Overall, the proposed improvement establishes a practical and extensible experimental framework that complements existing research on numerical nondeterminism while providing a stronger empirical foundation for future investigations into reliable and reproducible LLM inference.

References

[1] H. Liu *et al*., “Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[2] D. Goldberg, “What Every Computer Scientist Should Know About Floating-Point Arithmetic,” *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[4] A. Vaswani *et al*., “Attention Is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5998–6008, 2017.

[5] P. Micikevicius *et al*., “Mixed Precision Training,” *International Conference on Learning Representations (ICLR)*, 2018.

[6] A. Paszke *et al*., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[8] Hugging Face, “Transformers Documentation,” 2025.

[9] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*. New York, NY, USA: IEEE, 2019.

[10] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed., Draft, 2025.

APPENDICES

Appendix A – Evaluation Prompts

This appendix presents the prompts used to evaluate the deterministic behaviour of the TinyLlama language model. The prompts were selected to assess consistency across repeated executions while representing different natural language tasks.

Prompt ID	Prompt
P1	Explain deterministic inference
P2	What are the benefits of reproducible AI
P3	Describe greedy decoding
P4	Explain deterministic output in healthcare

Appendix B – Experimental Environment

Table B.1 summarises the software and hardware environment used during the evaluation.

Component	Specification
Operating System	Windows 11
Programming Language	Python 3.x
Framework	PyTorch
Transformers Library	Hugging Face Transformers

Model	TinyLlama-1.1B-Chat-v1.0
IDE	Visual Studio Code / Jupyter Notebook
Data Processing	pandas
Visualisation	matplotlib

Appendix C – Project Structure

The project was organised into a modular architecture to improve maintainability and readability.

```
LLM-Nondeterminism-Improvement-main/
|
├── README.md
├── requirements.txt
├── LICENSE
├── notebook/
├── src/
|   ├── config.py
|   ├── model_loader.py
|   ├── inference.py
|   ├── prompts.py
|   ├── metrics.py
|   ├── evaluation.py
|   ├── visualization.py
|   ├── export_results.py
|   └── main.py
├── images/
├── results/
└── report/
```

Appendix D – Sample Experimental Output

Deterministic inference is a decoding strategy in which identical inputs consistently produce identical outputs by removing stochastic sampling during text generation.

Inference Time

2.81 seconds

GPU Memory Usage

980 MB

Appendix E – Repository Information

The link to the repository will be included in the Part 2 Appendix section